

Exercício 1 — Classe Abstrata Base

Crie uma **classe abstrata** chamada **Pessoa** que servirá de base para outras classes.

Estrutura esperada:

```
<?php
abstract class Pessoa {
    protected $nome;
    protected $idade;
    protected $sexo;

    public function __construct($nome, $idade, $sexo) {
        $this->nome = $nome;
        $this->idade = $idade;
        $this->sexo = $sexo;
    }

    // Método comum (não abstrato)
    final public function fazerAniversario() {
        $this->idade++;
        echo "<p>Parabéns, {$this->nome}! Agora você tem
{$this->idade} anos.</p>";
    }

    // Método abstrato
    abstract public function apresentar();
}
```

Tarefas:

1. Crie a classe acima.
2. Explique por que não é possível instanciá-la diretamente (`new Pessoa()` deve gerar erro). Porque é uma classe abstrata, serve apenas de modelo para outras classes.
3. Tentar fazer `new Pessoa()` gera erro: Não é possível instanciar a classe abstrata Pessoa.
4. Identifique o papel do método **final**. O método final não pode ser sobrescrito por nenhuma subclasse. Assim, todas as classes que herdarem de Pessoa terão o mesmo comportamento para `fazerAniversario()`.

5. Explique a função de um método **abstrato**. Obriga cada subclasse de *Pessoa* a definir sua própria versão desse método, garantindo comportamento personalizado.

🧩 Exercício 2 — Herança de Implementação

Crie uma classe chamada *Visitante* que herda de *Pessoa* e implementa o método abstrato *apresentar()*.

Estrutura esperada:

```
class Visitante extends Pessoa {
    public function apresentar() {
        echo "<p>Sou um visitante chamado {$this->nome}.</p>";
    }
}
```

Tarefas:

1. Implemente *Visitante*.
2. Instancie um visitante e teste os métodos herdados (*fazerAniversario()* e *apresentar()*).

```
<?php

class Visitante extends Pessoa {

    public function apresentar() {

        echo "<p>Sou um visitante chamado {$this->nome}.</p>";

    }

}

// Teste

$v = new Visitante("João", 30, "M");

$v->apresentar();

$v->fazerAniversario();

?>
```

3. Confirme que **Visitante** é uma **classe concreta** (instanciável). Visitante herda todos os atributos e métodos de Pessoa. É uma classe concreta, ou seja, pode ser instanciada porque implementa o método abstrato apresentar().
4. Essa herança é **pobre** ou **por diferença**? Justifique. Herança pobre (ou herança de implementação). Isso porque a subclasse não adiciona novos atributos nem comportamentos, apenas implementa o que já foi definido.

Exercício 3 — Herança por Diferença

Crie uma classe **Aluno** que também herda de **Pessoa**, mas **acrescenta novos atributos e métodos**.

Estrutura esperada:

```
class Aluno extends Pessoa {
    protected $matricula;
    protected $curso;

    public function __construct($nome, $idade, $sexo, $matricula,
    $curso) {
        parent::__construct($nome, $idade, $sexo);
        $this->matricula = $matricula;
        $this->curso = $curso;
    }

    public function apresentar() {
        echo "<p>Sou o aluno {$this->nome}, do curso de
    {$this->curso}</p>";
    }

    public function pagarMensalidade() {
        echo "<p>Mensalidade de {$this->nome} paga com
    sucesso!</p>";
    }
}
```

Tarefas:

1. Implemente **Aluno** conforme o modelo.

```
<?php

class Aluno extends Pessoa {

    protected $matricula;

    protected $curso;

    public function __construct($nome, $idade, $sexo, $matricula,
$curso) {

        parent::__construct($nome, $idade, $sexo);

        $this->matricula = $matricula;

        $this->curso = $curso;

    }

    public function apresentar() {

        echo "<p>Sou o aluno {$this->nome}, do curso de
{$this->curso}.</p>";

    }

    public function pagarMensalidade() {

        echo "<p>Mensalidade de {$this->nome} paga com sucesso!</p>";

    }

}

// Teste

$a = new Aluno("Maria", 20, "F", 1234, "Informática");
```

```
$a->apresentar();  
  
$a->fazerAniversario();  
  
$a->pagarMensalidade();  
  
?>
```

2. Explique o uso de `parent::__construct()`. chama o construtor da classe mãe (Pessoa), inicializando corretamente os atributos herdados.
3. Teste a criação de um objeto e verifique o método `fazerAniversario()` herdado. O método `fazerAniversario()` foi herdado e pode ser usado.
4. Por que `fazerAniversario()` não pode ser sobrescrito? Não pode ser sobrescrito porque é final.



Exercício 4 — Subclasse com Sobrescrita e Novo Método

Crie uma classe `Bolsista` que herda de `Aluno`, adicionando um atributo e sobrescrevendo um método.

Estrutura esperada:

```
class Bolsista extends Aluno {  
    private $bolsa;  
  
    public function __construct($nome, $idade, $sexo, $matricula,  
$curso, $bolsa) {  
        parent::__construct($nome, $idade, $sexo, $matricula,  
$curso);  
        $this->bolsa = $bolsa;  
    }  
  
    public function renovarBolsa() {  
        echo "<p>Bolsa renovada para {$this->nome}</p>";  
    }  
}
```

```
        public function pagarMensalidade() {
            echo "<p>{$this->nome} é bolsista! Pagamento com desconto de
{$this->bolsa}%.</p>";
        }
    }
}
```

Tarefas:

1. Implemente **Bolsista** e teste os métodos.

```
<?php

class Bolsista extends Aluno {

    private $bolsa;

    public function __construct($nome, $idade, $sexo, $matricula,
$curso, $bolsa) {

        parent::__construct($nome, $idade, $sexo, $matricula,
$curso);

        $this->bolsa = $bolsa;

    }

    public function renovarBolsa() {

        echo "<p>Bolsa renovada para {$this->nome}!</p>";

    }

    // Sobrescrita do método

    public function pagarMensalidade() {

        echo "<p>{$this->nome} é bolsista! Pagamento com desconto
de {$this->bolsa}%.</p>";

    }

}
```

```

}

// Teste

$b = new Bolsista("Lucas", 19, "M", 5678, "Design", 50);

$b->apresentar();

$b->pagarMensalidade();

$b->renovarBolsa();

$b->fazerAniversario();

?>

```

2. Identifique o conceito de **polimorfismo** no método `pagarMensalidade()`. O método `pagarMensalidade()` é sobrescrito, ou seja, tem o mesmo nome da classe mãe (Aluno), mas um comportamento diferente. Isso é polimorfismo, pois o mesmo método age de maneira distinta dependendo da classe.
3. Análise: o método `fazerAniversario()` pode ser sobrescrito? Por quê? O método `fazerAniversario()` não pode ser sobrescrito porque é final.

Exercício 5 — Classe Final e Hierarquia Completa

Crie uma classe `Professor` que herda de `Pessoa`, e declare-a como **final**, impedindo novas heranças.

Estrutura esperada:

```

final class Professor extends Pessoa {
    private $especialidade;
    private $salario;

    public function __construct($nome, $idade, $sexo, $esp,
    $salario) {
        parent::__construct($nome, $idade, $sexo);
        $this->especialidade = $esp;
        $this->salario = $salario;
    }

    public function apresentar() {

```

```

        echo "<p>Sou o professor {$this->nome}, especialista em
{$this->especialidade}</p>";
    }

    public function receberAumento($valor) {
        $this->salario += $valor;
        echo "<p>O salário de {$this->nome} foi reajustado para R$
{$this->salario}</p>";
    }
}

```

Tarefas:

1. Crie a classe `Professor` com `final`.
2. Tente criar uma classe `Coordenador` `extends Professor` e observe o erro.
3. Crie um vetor com um `Visitante`, um `Aluno`, um `Bolsista` e um `Professor`.
4. Use `get_class($obj)` e `instanceof` para identificar a hierarquia.

`final` class `Professor` impede que outras classes herdem dela. Isso garante que o comportamento de `Professor` não será alterado por subclasses. `get_class()` mostra o nome da classe concreta. `instanceof` verifica se o objeto pertence (ou herda) de uma classe específica.

```

<?php

final class Professor extends Pessoa {

    private $especialidade;

    private $salario;

    public function __construct($nome, $idade, $sexo, $esp,
$salario) {

        parent::__construct($nome, $idade, $sexo);

        $this->especialidade = $esp;

        $this->salario = $salario;
    }
}

```



```

    }

    public function apresentar() {

        echo "<p>Sou o professor {$this->nome}, especialista em
{$this->especialidade}</p>";

    }

    public function receberAumento($valor) {

        $this->salario += $valor;

        echo "<p>O salário de {$this->nome} foi reajustado para R$
{$this->salario}</p>";

    }
}

// Tentativa de herança (gera erro):

// class Coordenador extends Professor {} x

// Erro: "Class Coordenador may not inherit from final class
(Professor) "

// Testando a hierarquia

$visitante = new Visitante("Ana", 25, "F");

$aluno = new Aluno("Bruno", 21, "M", 1111, "TI");

$bolsista = new Bolsista("Carla", 22, "F", 2222, "Engenharia",
75);

$professor = new Professor("Claiton", 40, "M", "Programação",
5000);

```

```
$vetor = [$visitante, $aluno, $bolsista, $professor];

foreach ($vetor as $obj) {

    echo "<hr>";

    echo "<p>Classe: " . get_class($obj) . "</p>";

    if ($obj instanceof Pessoa) {

        echo "<p>É um tipo de Pessoa.</p>";

    }

    $obj->apresentar();

}

?>
```

5. Identifique qual é a **raiz** e quais são as **folhas** da árvore de herança.

Raiz da hierarquia: Pessoa

Folhas da hierarquia: Visitante, Bolsista, Professor (não possuem subclasses).